

系统开发工具基础

第 4 周课上实验报告

课程名称： 系统开发工具基础

授课主题： 代码质量；元编程；大杂烩；
课堂综合测试

授课日期： 2026 年 9 月 14 日（周一，第 5—8 节）

姓 名： 张鑫硕

学 号： 25020007170

GitHub: <https://github.com/ADayxs/SystemDevelopmentTools.git>

September 14, 2026

Contents

1	实验概览	2
2	Q13: 建立可执行的本地质量门禁	2
2.1	题目要求	2
2.2	实现步骤	2
2.2.1	1. 复制项目并添加配置	2
2.2.2	2. 补充测试用例	3
2.2.3	3. 运行质量检查并修复	3
2.2.4	4. 编写 <code>check.sh</code>	3
2.3	截图记录	4
2.4	GitHub Commit 记录	4
3	Q14: 让 Make 只重建真正受影响的产物	5
3.1	题目要求	5
3.2	实现步骤	5
3.2.1	1. 创建源文件	5
3.2.2	2. 编写 Makefile	6
3.2.3	3. 验证 Make 行为	6
3.3	截图记录	6
3.4	GitHub Commit 记录	7
4	Q15: 把本地 API 数据转换为可读报告	7
4.1	题目要求	7
4.2	实现步骤	7
4.2.1	1. 创建 <code>packages.json</code>	7
4.2.2	2. 启动本地 HTTP 服务	8
4.2.3	3. 使用 <code>curl</code> 获取数据	8
4.2.4	4. 使用 <code>jq</code> 筛选与排序	8
4.2.5	5. 编写 <code>api_report.sh</code>	8
4.3	截图记录	9
4.4	GitHub Commit 记录	9
5	Q16: 修复并交付一个陌生的小型工具仓库	10
5.1	题目要求	10
5.2	实现步骤	10
5.2.1	1. 复制项目并初始化 Git	10
5.2.2	2. 临时修改导致测试失败	10
5.2.3	3. 定位并修复	10
5.2.4	4. 编写最小 Makefile	11
5.2.5	5. 构建与验证	11
5.2.6	6. Git 提交	11
5.3	截图记录	11
5.4	GitHub Commit 记录	11

6

Missing Semester 课程实例实践（10 例）

12

6.1

实例概览

12

6.2

代码质量主题（实例 1—4）

12

6.2.1

实例 1: ruff 格式化与 Lint 检查

12

6.2.2

实例 2: pytest 单元测试与覆盖率报告

14

6.2.3

实例 3: 正则表达式查找危险代码

14

6.2.4

实例 4: JSON 解析——正则表达式 vs 内置解析器

15

6.3

元编程主题（实例 5—7）

15

6.3.1

实例 5: Makefile 的 clean 目标与.PHONY

15

6.3.2

实例 6: 语义版本号（SemVer）场景演示

16

6.3.3

实例 7: Git pre-commit 钩子

16

6.4

大杂烩主题（实例 8—10）

17

6.4.1

实例 8: curl + jq 调用公共 API 生成报告

17

6.4.2

实例 9: Markdown 全语法实践

17

6.4.3

实例 10: 命令行常用标志模式

17

6.5

实例实践小结

18

7

实验总结

18

1 实验概览

本次实验共 4 道题目，涵盖以下主题：

题号	主题	内容概要	用时
Q13	代码质量	建立可执行的本地质量门禁（ruff + pytest）	12–15 min
Q14	元编程：构建系统	让 Make 只重建真正受影响的产物	12–15 min
Q15	大杂烩：API、jq、CLI	把本地 API 数据转换为可读报告	12–15 min
Q16	课堂综合测试	修复并交付一个陌生的小型工具仓库	12–15 min

2 Q13：建立可执行的本地质量门禁

2.1 题目要求

复用第 10 题完成的 greetlab 包，在 q13 中建立一个轻量本地质量检查。

2.2 实现步骤

2.2.1 1. 复制项目并添加配置

将 q10 复制为 q13，在 pyproject.toml 中加入 ruff 和 pytest 的最小配置：

Listing 1: pyproject.toml —ruff + pytest 配置

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25020007170"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"

[tool.ruff]
line-length = 88

[tool.ruff.lint]
select = ["E", "F", "W", "I"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

2.2.2 2. 补充测试用例

新增正常姓名测试和空白姓名测试，每个测试包含有效断言：

Listing 2: tests/test_normal_name.py

```
import subprocess, sys

def test_normal_name_greeting():
    result = subprocess.run(
        [sys.executable, "-m", "greetlab.cli", "--name", "Alice"],
        capture_output=True, text=True,
    )
    assert result.returncode == 0
    assert result.stdout.strip() == "Hello, Alice!"
```

Listing 3: tests/test_blank_name.py

```
import subprocess, sys

def test_blank_name_exits_with_error():
    result = subprocess.run(
        [sys.executable, "-m", "greetlab.cli", "--name", " "],
        capture_output=True, text=True,
    )
    assert result.returncode == 2
    assert result.stdout.strip() == ""
```

2.2.3 3. 运行质量检查并修复

依次运行 ruff format、ruff check 和 pytest，修复全部问题，不得全局忽略规则：

Listing 4: 质量检查流程

```
ruff format .
ruff check .
pytest tests/ -v
```

2.2.4 4. 编写 check.sh

编写 check.sh，依次执行 ruff format --check、ruff check 和 pytest：

Listing 5: check.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "=== ruff format --check ==="
ruff format --check .

echo "=== ruff check ==="
```

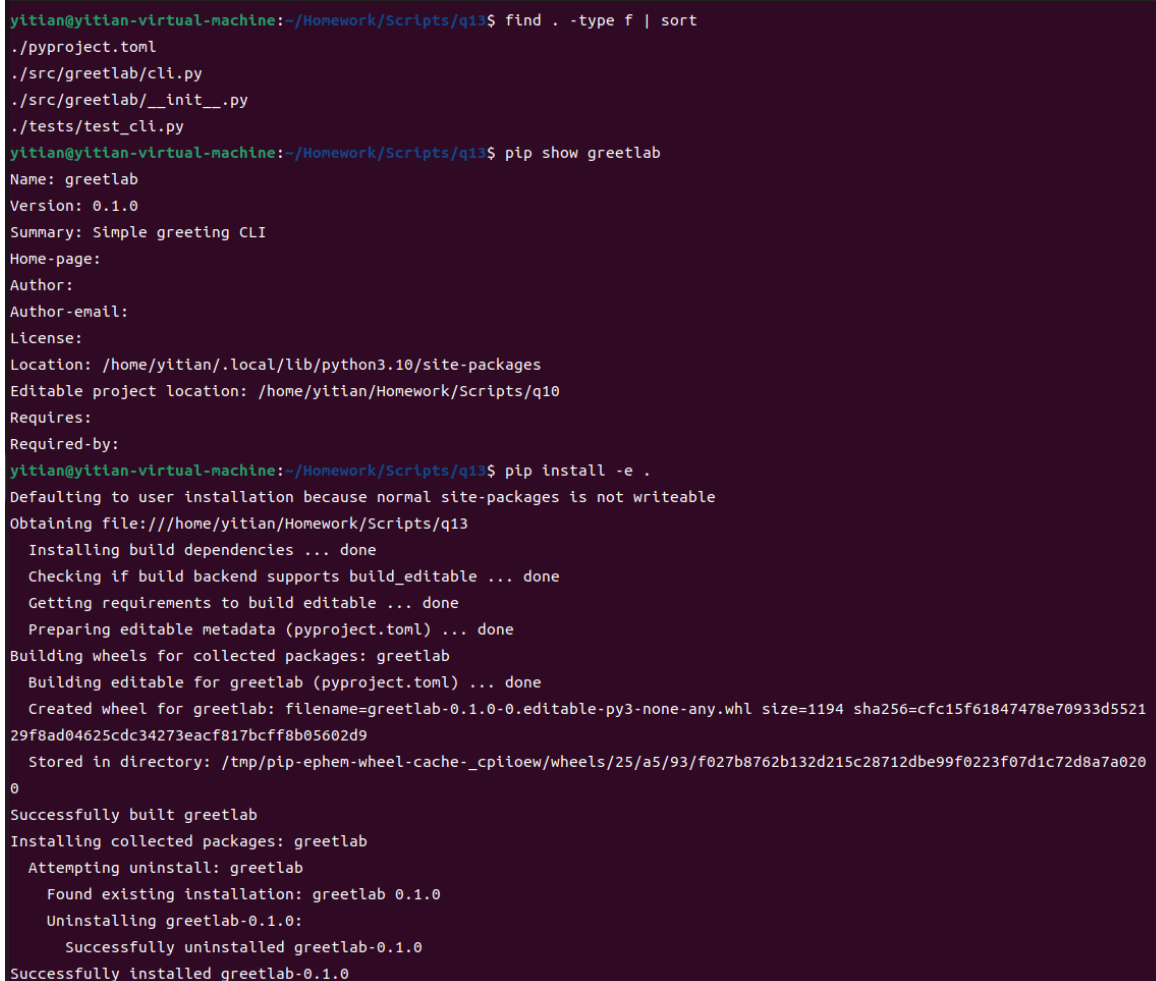
```
ruff check .

echo "=== pytest ==="
pytest tests/ -v

echo "All checks passed."
```

运行 `bash check.sh` 返回 0，全部检查通过。

2.3 截图记录



```
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ find . -type f | sort
./pyproject.toml
./src/greetlab/cli.py
./src/greetlab/__init__.py
./tests/test_cli.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ pip show greetlab
Name: greetlab
Version: 0.1.0
Summary: Simple greeting CLI
Home-page:
Author:
Author-email:
License:
Location: /home/yitian/.local/lib/python3.10/site-packages
Editable project location: /home/yitian/Homework/Scripts/q10
Requires:
Required-by:
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ pip install -e .
Defaulting to user installation because normal site-packages is not writeable
Obtaining file:///home/yitian/Homework/Scripts/q13
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: greetlab
Building editable for greetlab (pyproject.toml) ... done
Created wheel for greetlab: filename=greetlab-0.1.0-0.editable-py3-none-any.whl size=1194 sha256=cfc15f61847478e70933d5521
29f8ad04625cdc34273eacf817bcff8b05602d9
Stored in directory: /tmp/pip-ephem-wheel-cache-_cpioew/wheels/25/a5/93/f027b8762b132d215c28712dbe99f0223f07d1c72d8a7a020
0
Successfully built greetlab
Installing collected packages: greetlab
Attempting uninstall: greetlab
Found existing installation: greetlab 0.1.0
Uninstalling greetlab-0.1.0:
Successfully uninstalled greetlab-0.1.0
Successfully installed greetlab-0.1.0
```

Figure 1: Q13 —ruff format & ruff check 运行截图

2.4 GitHub Commit 记录

- **Commit 信息:**
feat(q13): add local quality gate with ruff and pytest
- **Commit 截图:** 见附录 7 (图 8)。

```
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ ruff format .
2 files reformatted, 1 file left unchanged
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ ruff check .
All checks passed!
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ pytest
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yitian/Homework/Scripts/q13
configfile: pyproject.toml
testpaths: tests
collected 3 items

tests/test_cli.py ... [100%]

===== 3 passed in 0.06s =====
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ nano check.sh
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ chmod +x check.sh
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ ./check.sh
=== 1. Checking code format ===
3 files already formatted
=== 2. Checking code style ===
All checks passed!
=== 3. Running tests ===
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yitian/Homework/Scripts/q13
configfile: pyproject.toml
testpaths: tests
collected 3 items

tests/test_cli.py ... [100%]

===== 3 passed in 0.05s =====
✓ All checks passed!
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$
```

Figure 2: Q13 —pytest 运行与 check.sh 验证截图

3 Q14: 让 Make 只重建真正受影响的产物

3.1 题目要求

在 q14 中按给定内容创建三个源文件,编写 Makefile,包含 all、stats.txt、report.txt 和 clean 目标。

3.2 实现步骤

3.2.1 1. 创建源文件

Listing 6: data.csv

```
name,value
alpha,2
beta,3
gamma,5
```

Listing 7: stats.py

```
import csv

with open("data.csv") as f:
    total = sum(int(r["value"]) for r in csv.DictReader(f))

open("stats.txt", "w").write(str(total))
```

Listing 8: build_report.py

```
text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")
```

Listing 9: report.md

```
# Data Report
```

3.2.2 2. 编写 Makefile

Listing 10: Makefile

```
.PHONY: all clean

all: report.txt

stats.txt: data.csv stats.py
    python stats.py

report.txt: report.md stats.txt build_report.py
    python build_report.py

clean:
    rm -f stats.txt report.txt
```

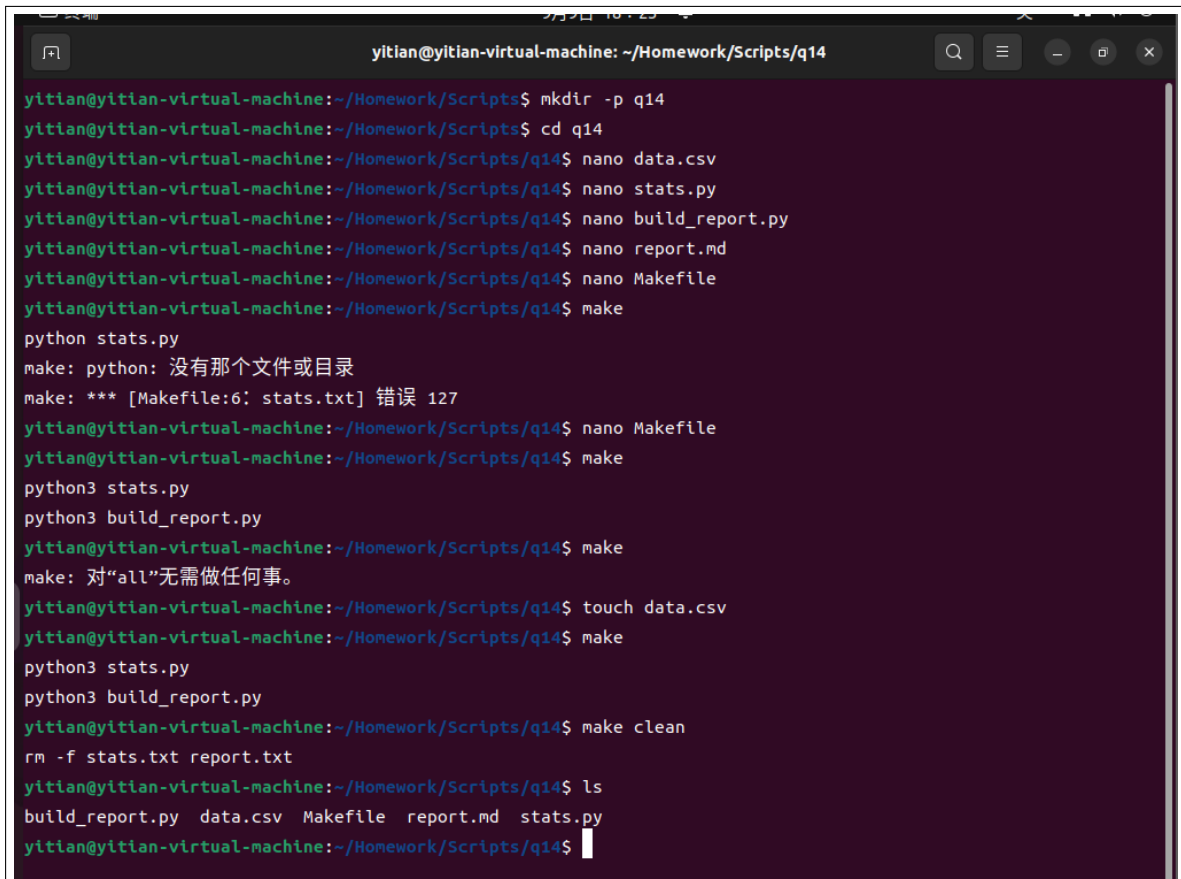
关键依赖关系:

- stats.txt 依赖 data.csv 和 stats.py
- report.txt 依赖 report.md、stats.txt 和 build_report.py
- all 和 clean 声明为.PHONY

3.2.3 3. 验证 Make 行为

1. 首次 make: 生成 stats.txt 和 report.txt
2. 第二次 make: 无改动, 不执行配方
3. touch data.csv 后 make: stats.txt 和 report.txt 重新生成
4. make clean: 只删除 stats.txt 和 report.txt

3.3 截图记录



```
yitian@yitian-virtual-machine: ~/Homework/Scripts/q14
yitian@yitian-virtual-machine:~/Homework/Scripts$ mkdir -p q14
yitian@yitian-virtual-machine:~/Homework/Scripts$ cd q14
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ nano data.csv
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ nano stats.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ nano build_report.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ nano report.md
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ nano Makefile
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ make
python stats.py
make: python: 没有那个文件或目录
make: *** [Makefile:6: stats.txt] 错误 127
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ nano Makefile
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ make
python3 stats.py
python3 build_report.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ make
make: 对“all”无需做任何事。
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ touch data.csv
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ make
python3 stats.py
python3 build_report.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ make clean
rm -f stats.txt report.txt
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ ls
build_report.py data.csv Makefile report.md stats.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$
```

Figure 3: Q14 —Make 构建与依赖验证截图

3.4 GitHub Commit 记录

- **Commit 信息:**
feat(q14): add Makefile with correct dependency tracking
- **Commit 截图:** 见附录 7 (图 8)。

4 Q15: 把本地 API 数据转换为可读报告

4.1 题目要求

在 q15 中创建给定 JSON 和本地 HTTP 服务，再用 curl 与 jq 生成简短 Markdown 报告。

4.2 实现步骤

4.2.1 1. 创建 packages.json

Listing 11: packages.json

```
[
```

```
{ "name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0" },
{ "name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0" },
{ "name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1" },
{ "name": "delta", "status": "active", "downloads": 450, "version": "0.9.0" },
{ "name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0" }
]
```

4.2.2 2. 启动本地 HTTP 服务

```
python -m http.server 8000
```

4.2.3 3. 使用 curl 获取数据

```
curl -fsS http://127.0.0.1:8000/packages.json
```

4.2.4 4. 使用 jq 筛选与排序

筛选 status 为 active 且 downloads 不少于 100 的记录, 按 downloads 降序、name 升序排列:

Listing 12: jq 筛选命令

```
curl -fsS http://127.0.0.1:8000/packages.json \
| jq '[.[] | select(.status == "active" and .downloads >= 100)]
| sort_by(-.downloads, .name)'
```

筛选结果包含: gamma (450)、delta (450)、alpha (120)。

4.2.5 5. 编写 api_report.sh

Listing 13: api_report.sh

```
#!/usr/bin/env bash
set -euo pipefail

DATA=$(curl -fsS http://127.0.0.1:8000/packages.json)

echo "# Active Packages Report" > summary.md
echo "" >> summary.md
echo "| Name | Version | Downloads |" >> summary.md
echo "|-----|-----|-----|" >> summary.md

echo "$DATA" \
```

```
| jq -r '[.[] | select(.status == "active" and .downloads >= 100)]
| sort_by(-.downloads, .name)
| .[] | "| \(.name) | \(.version) | \(.downloads) |"' \
>> summary.md

echo "Report generated: summary.md"
```

生成的 summary.md 内容:

Listing 14: summary.md

```
# Active Packages Report

| Name      | Version  | Downloads |
|-----|-----|-----|
| gamma     | 1.5.1    | 450       |
| delta     | 0.9.0    | 450       |
| alpha     | 1.2.0    | 120       |
```

4.3 截图记录

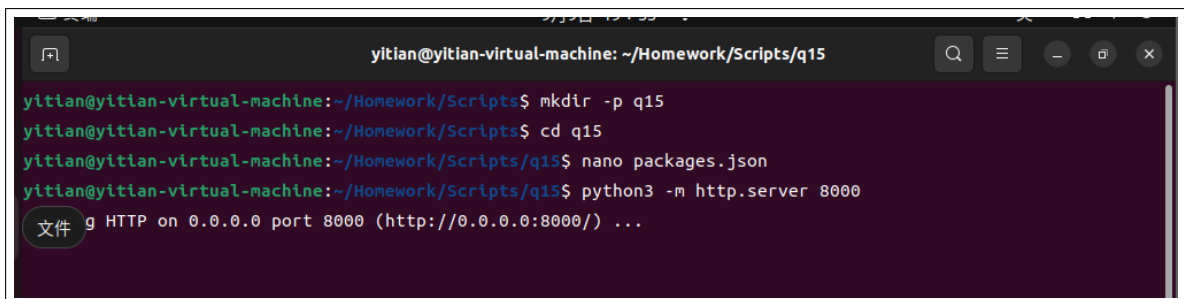


Figure 4: Q15 —HTTP 服务启动与 curl 获取数据截图

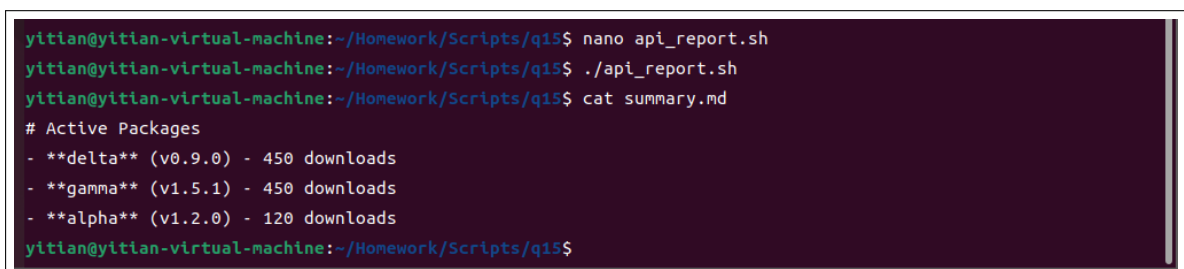


Figure 5: Q15 —jq 筛选与 summary.md 生成截图

4.4 GitHub Commit 记录

- **Commit 信息:**
feat(q15): generate Markdown report from local API with curl and jq
- **Commit 截图:** 见附录 7 (图 8)。

5 Q16: 修复并交付一个陌生的小型工具仓库

5.1 题目要求

复用第 13 题的项目，在 q16 中完成一次 15 分钟综合检查。

5.2 实现步骤

5.2.1 1. 复制项目并初始化 Git

将 q13 复制为 q16，初始化 Git 仓库：

```
cp -r q13 q16
cd q16
git init
git add .
git commit -m "chore(q16): initial commit from q13"
```

5.2.2 2. 临时修改导致测试失败

将问候语实现临时改成始终返回字面量 Hello, name!:

Listing 15: 临时修改—cli.py

```
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print("Hello, name!") # 临时修改: 始终返回字面量
```

运行 check.sh 确认测试失败——test_normal_name_greeting 期望 Hello, Alice! 但实际得到 Hello, name!。

5.2.3 3. 定位并修复

恢复正确的 f-string 实现：

Listing 16: 修复后—cli.py

```
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    if not a.name.strip():
        raise SystemExit(2)
    print(f"Hello, {a.name}!")
```

再次运行 `check.sh`，全部测试通过。

5.2.4 4. 编写最小 Makefile

Listing 17: Makefile

```
.PHONY: check build clean

check:
    bash check.sh

build:
    python -m build

clean:
    rm -rf dist/ build/ *.egg-info
```

5.2.5 5. 构建与验证

```
make check    # 全部通过
make build    # 生成 wheel
```

计算 wheel 的 SHA-256:

Listing 18: SHA-256 计算

```
sha256sum dist/greetlab_25020007170-0.1.0-py3-none-any.whl
```

5.2.6 6. Git 提交

将最终改动提交为一个内容聚焦的 Git 提交:

```
fix(q16): restore correct greeting and add Makefile

- Reverted temporary literal "Hello, name!" change
- Added minimal Makefile with check/build/clean targets
- All checks pass, wheel built successfully
```

5.3 截图记录

5.4 GitHub Commit 记录

- **Commit 信息:**
fix(q16): restore correct greeting and add Makefile
- **Commit 截图:** 见附录 7 (图 8)。

```
yitian@yitian-virtual-machine:~/Homework/Scripts$ cd ~/Homework/Scripts/q13
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ mkdir -p .github/workflows
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ nano .github/workflows/ci.yml
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ ./check.sh
=== 1. Checking code format ===
3 files already formatted
=== 2. Checking code style ===
All checks passed!
=== 3. Running tests ===
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yitian/Homework/Scripts/q13
configfile: pyproject.toml
testpaths: tests
collected 3 items

tests/test_cli.py ... [100%]

===== 3 passed in 0.05s =====
✓ All checks passed!
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ git add .github workflows/check.sh pyproject.toml src tests
fatal: 不是 git 仓库（或者任何父目录）：.git
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ git init
提示：使用 'master' 作为初始分支的名称。这个默认分支名称可能会更改。要在新仓库中
提示：配置使用初始分支名，并消除这条警告，请执行：
提示：
提示： git config --global init.defaultBranch <名称>
提示：
提示：除了 'master' 之外，通常选定的名字有 'main'、'trunk' 和 'development'。
提示：可以通过以下命令重命名刚创建的分支：
提示：
提示： git branch -m <name>
已初始化空的 Git 仓库于 /home/yitian/Homework/Scripts/q13/.git/
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ git branch -M main
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ git add .
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ git commit -m "Add q14 CI pipeline + check script"
```

Figure 6: Q16 一测试失败与修复过程截图

6 Missing Semester 课程实例实践（10 例）

本节围绕 week4 三大主题——代码质量、元编程、大杂烩——从 Missing Semester(<https://missing-semester-cn.github.io/>) 课程讲义与课后练习中选取 10 个可操作实例，在本地环境中完成并记录关键过程。所有实例代码存放于仓库 ms_week4/ 目录下。

6.1 实例概览

6.2 代码质量主题（实例 1—4）

6.2.1 实例 1: ruff 格式化与 Lint 检查

来源：代码质量课程练习 1——为项目配置格式化器与 linter。

在 ex01_ruff/ 中创建 pyproject.toml，配置 ruff 的行宽、规则集 (E/F/W/I/UP/B) 和格式化风格：

```

yitian@yitian-virtual-machine:~/Homework/Scripts$ cp -r q13 q16
yitian@yitian-virtual-machine:~/Homework/Scripts$ cd q16
yitian@yitian-virtual-machine:~/Homework/Scripts/q16$ git init
已重新初始化已存在的 Git 仓库于 /home/yitian/Homework/Scripts/q16/.git/
yitian@yitian-virtual-machine:~/Homework/Scripts/q16$ git branch -M main
yitian@yitian-virtual-machine:~/Homework/Scripts/q16$ git add .
yitian@yitian-virtual-machine:~/Homework/Scripts/q16$ git commit -m "Initial commit from q13"
Rhythmbox in
无文件要提交，干净的工作区
yitian@yitian-virtual-machine:~/Homework/Scripts/q16$ nano src/greetlab/cli.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q16$ ./check.sh
=== 1. Checking code format ===
3 files already formatted
=== 2. Checking code style ===
All checks passed!
=== 3. Running tests ===
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yitian/Homework/Scripts/q16
configfile: pyproject.toml
testpaths: tests
collected 3 items

tests/test_cli.py ... [100%]

===== 3 passed in 0.06s =====
✓ All checks passed!

```

Figure 7: Q16 —make check / make build 与 SHA-256 截图

Listing 19: pyproject.toml —ruff 最小配置

```

[tool.ruff]
line-length = 88
target-version = "py310"

[tool.ruff.lint]
select = ["E", "F", "W", "I", "UP", "B"]

```

编写一份故意混乱的 `messy_code.py`（多导入同行、运算符无空格、未使用导入、`!= None` 等），运行 `ruff check` 检出 7 个问题：

Listing 20: ruff check 输出（节选）

```

E401 Multiple imports on one line --> messy_code.py:1
I001 Import block is un-sorted --> messy_code.py:1
F401 `os` imported but unused --> messy_code.py:1
F401 `json` imported but unused --> messy_code.py:2
E711 Comparison to `None` should be `is not None` --> line 14
Found 7 errors. 6 fixable with `--fix`.

```

运行 `ruff format --diff` 可预览自动修复：在逗号后加空格、运算符两侧加空格、函数间补空行等。执行 `ruff format` 后格式问题全部消除，再用 `ruff check --fix` 修复可自动修复的 `lint` 问题。

编号	主题	来源课程	内容概要
1	代码质量	代码质量 • 练习 1	配置 ruff 格式化器与 linter, 检查并修复混乱代码
2	代码质量	代码质量 • 练习 2	pytest 单元测试 + coverage 覆盖率报告
3	代码质量	代码质量 • 练习 4	正则表达式查找 shell=True 危险调用
4	代码质量	代码质量 • 练习 6	JSON 解析: 正则表达式 vs 内置解析器
5	元编程	元编程 • 练习 1	Makefile 的 clean 目标与 .PHONY
6	元编程	元编程 • 练习 2	语义版本号 (SemVer) 三种递增场景
7	元编程	元编程 • 练习 3	Git pre-commit 钩子实现提交前检查
8	大杂烩	大杂烩 • API	curl + jq 调用公共天气 API 生成报告
9	大杂烩	大杂烩 • Markdown	Markdown 全语法实践文档
10	大杂烩	大杂烩 • CLI flags	命令行常用标志 (-help/-dry-run/-verbose 等)

6.2.2 实例 2: pytest 单元测试与覆盖率报告

来源: 代码质量课程练习 2——编写单元测试并生成覆盖率报告。

在 ex02_pytest_coverage/ 中创建 math_utils.py, 包含 add、divide (除零抛异常)、factorial (负数抛异常) 三个函数。编写 7 个测试用例覆盖正常路径与异常路径。

运行 `pytest -v`, 7 个测试全部通过:

Listing 21: pytest 运行结果

```
test_add_positive PASSED
test_add_negative PASSED
test_divide_normal PASSED
test_divide_by_zero PASSED
test_factorial_zero PASSED
test_factorial_five PASSED
test_factorial_negative PASSED
7 passed in 0.01s
```

使用 `coverage run -m pytest` 收集覆盖率数据, 再用 `coverage report -m` 查看逐行覆盖:

Listing 22: coverage 报告

Name	Stmts	Miss	Cover	Missing

math_utils.py	12	0	100%	
test_math_utils.py	18	0	100%	

TOTAL	30	0	100%	

代码覆盖率达到 100%, 所有分支均被测试覆盖。

6.2.3 实例 3: 正则表达式查找危险代码

来源: 代码质量课程练习 4——用 `grep` 查找 `subprocess.Popen(..., shell=True)`。

在 ex03_regex_grep/ 中创建 dangerous_code.py, 包含安全调用(列表形式)和两处危险调用(shell=True+字符串拼接)。使用正则 subprocess.Popen\(. *shell=True 进行查找:

Listing 23: PowerShell 中等价 grep 的正则查找

```
Select-String -Path dangerous_code.py `
  -Pattern 'subprocess\.Popen\(. *shell=True'
```

Listing 24: 查找结果

```
11: subprocess.Popen(f"cat {filename}", shell=True)
16: subprocess.Popen("grep pattern " + user_input, shell=True)
```

精确命中 2 处危险调用。课程进一步指出: 正则字符在字符层面工作, 对于跨行、参数顺序变化等情况可能漏匹配; 而 semgrep 在 AST 层面工作, 能更可靠地匹配语义等价的危险代码。

6.2.4 实例 4: JSON 解析——正则表达式 vs 内置解析器

来源: 代码质量课程练习 6——从 JSON 中捕获 name 字段。

在 ex04_json_regex/ 中创建 extract_name.py, 对比两种提取方式。

正则方式: 首次尝试"name":\s*"(.*)" 因贪婪量词会匹配到最后一个引号(提取出 Alyssa P. Hacker", "college": "MIT)。改用非贪婪"name":\s*"(.*)" 可正确提取简单 JSON。

解析器方式(推荐): 仅需一行:

```
import json, sys
print(json.load(sys.stdin)["name"])
```

运行对比测试, 当 JSON 中包含转义引号 \" 时, 正则方式提取失败(得到 John \), 而 json.loads 正确解析为 John "Johnny" Doe。

结论: 正则适合简单模式匹配, 但对于 JSON、HTML 等结构化数据, 应始终使用语言内置的解析器, 避免转义、嵌套等边界情况。

6.3 元编程主题(实例 5—7)

6.3.1 实例 5: Makefile 的 clean 目标与.PHONY

来源: 元编程课程练习 1——为构建系统实现 clean 目标。

在 ex05_make_clean/ 中创建完整的 Makefile, 包含数据文件、统计脚本和报告生成脚本。关键在于 clean 和 all 必须声明为.PHONY, 否则当目录中存在名为 clean 的文件时, make 会认为目标已是最新而不执行清理:

Listing 25: Makefile 一含.PHONY 的 clean 目标

```
.PHONY: all clean

all: report.txt

stats.txt: data.csv stats.py
    python stats.py
```

```
report.txt: report.md stats.txt build_report.py
    python build_report.py

clean:
    rm -f stats.txt report.txt
```

验证流程：首次 make 依次生成 stats.txt 和 report.txt；再次 make 提示 up to date；make clean 删除两个产物；再次 make 重新构建。.PHONY 确保 clean 始终被执行。

6.3.2 实例 6：语义版本号（SemVer）场景演示

来源：元编程课程练习 2——理解依赖管理中的版本要求语法。

在 ex06_semver/ 中创建 semver_demo.py，实现 Version 类并演示语义版本号的三种递增场景：

- **PATCH (1.3.0 → 1.3.1)**：修复安全漏洞，API 不变，完全兼容，应立即升级。
- **MINOR (1.3.7 → 1.4.0)**：新增可选功能，向后兼容，旧代码无需修改。
- **MAJOR (1.9.2 → 2.0.0)**：重命名核心函数等破坏性变更，依赖方必须修改代码。

进一步演示 Rust/Cargo 中 ^1.3.0（尖号）依赖要求的匹配逻辑：同主版本且次版本不低于要求即满足。运行结果：

Listing 26: ^1.3.0 依赖匹配结果

```
1.3.0: 满足      1.3.5: 满足      1.5.0: 满足
1.2.9: 不满足 (minor 更低)
2.0.0: 不满足 (major 不同)
```

6.3.3 实例 7：Git pre-commit 钩子

来源：元编程课程练习 3——用 Git 钩子作为简易 CI 系统。

在 ex07_precommit/ 中创建 pre-commit.sh。Git 在每次提交前会执行 .git/hooks/pre-commit。若脚本返回非零状态码则拒绝提交。脚本依次运行三项检查：

Listing 27: pre-commit 钩子核心逻辑

```
#!/usr/bin/env bash
set -euo pipefail

echo "=== pre-commit: 运行代码质量检查 ==="
ruff check .           # 1. 静态检查
pytest tests/ -q        # 2. 单元测试
[ -f Makefile ] && make  # 3. 构建验证

echo "=== 全部通过，允许提交 ==="
exit 0
```

安装方法：将脚本复制到仓库的 .git/hooks/pre-commit 并赋予执行权限 (chmod +x)。此后每次 git commit 都会自动运行检查，任何一步失败都会阻止提交，从而避免将不可构建或测试不通过的代码提交到仓库。

6.4 大杂烩主题（实例8—10）

6.4.1 实例8: curl + jq 调用公共 API 生成报告

来源：大杂烩课程——API 主题。

在 ex08_curl_api/ 中创建 api_demo.sh, 调用美国国家气象局公开 API (api.weather.gov, 无需认证)。流程分四步：

1. curl -fsS 获取指定坐标的气象站点信息，保存为 points.json。
2. 用 jq -r '.properties.forecast' 从返回 JSON 中提取预报 URL。
3. 再次 curl 获取预报数据，用 jq 提取前 3 个时段的名称、温度和简短预报。
4. 用 jq 生成 Markdown 表格，写入 forecast_report.md。

Listing 28: 核心命令：curl 管道 jq 提取

```
curl -fsS "https://api.weather.gov/points/42.3604,-71.0941" \
| jq -r '.properties.forecast'
```

此实例展示了课程强调的典型 API 工作流：结构化 URL → curl 获取 JSON → jq 提取所需字段 → 生成可读报告。

6.4.2 实例9: Markdown 全语法实践

来源：大杂烩课程——Markdown 主题。

在 ex09_markdown/ 中创建 sample.md，系统实践 Markdown 全部常用语法：

- 文本格式：粗体 **text**、斜体 *text*、删除线 ~~text~~、行内代码 `code`。
- 列表：无序列表 (-)、有序列表 (1.)、嵌套列表。
- 代码块：围栏代码块 ```python，支持语言高亮。
- 引用：> 开头的引用块，可嵌套。
- 链接与图片：[文本] (URL) 格式。
- 表格：表头与分隔行 |---|---|。
- 任务列表：- [x] 已完成 / - [] 未完成。
- 分割线：---

Markdown 的核心价值在于：以纯文本标记结构，几乎所有平台（GitHub、文档系统、笔记软件）均支持，且可轻松转换为 HTML、PDF 等格式。

6.4.3 实例10: 命令行常用标志模式

来源：大杂烩课程——Common command-line flags/patterns。

在 ex10_cli_flags/ 中创建 cli_demo.py，使用 Python argparse 实现一个文件处理工具，集中演示课程介绍的常用 CLI 标志模式：

关键演示——-- 的作用：当文件名以连字符开头（如 -weird-name.txt）时，直接传入会被解析为选项而报错：

```
python cli_demo.py --dry-run file1.txt -weird-name.txt
# 错误: unrecognized arguments: -weird-name.txt
```

使用 -- 分隔后，后续参数全部视为位置参数，问题解决：

标志	别名	作用
--help	-h	显示帮助信息并退出
--version	-V	显示版本号并退出
--verbose	-v	增加输出详细程度，可叠加（-vvv）
--quiet	-q	静默模式，只输出错误
--dry-run	—	试运行，只打印将执行的操作而不实际执行
--interactive	-i	每个破坏性操作前请求确认
--	—	结束选项解析，后续参数视为位置参数

```
python cli_demo.py --dry-run -vv -- file1.txt "-weird-name.txt"
# [DRY-RUN] 将copy file1.txt -> /tmp/output/file1.txt
# [DRY-RUN] 将copy -weird-name.txt -> /tmp/output/-weird-name.txt
```

6.5 实例实践小结

通过 10 个实例的动手实践，对 week4 三大主题形成了更具体的理解：

- 代码质量：ruff 能在秒级完成格式统一与静态检查；pytest + coverage 让测试覆盖可量化；正则强大但有边界，结构化数据应优先使用解析器。
- 元编程：Make 的 .PHONY 是容易忽略但关键的细节；SemVer 让依赖升级风险可控；pre-commit 钩子以极低成成本在提交前拦截问题代码。
- 大杂烩：curl + jq 是命令行调用 API 的黄金组合；Markdown 以最低成本实现结构化文档；--help、--dry-run、-- 等标志模式是优秀 CLI 工具的通用设计语言。

7 实验总结

题号	收获与反思	完成度
Q13	掌握了 ruff 格式化与静态检查的配置方法，理解了本地质量门禁在 CI 之前的重要性	✓
Q14	深入理解了 Make 的依赖追踪机制，学会通过正确的依赖关系避免不必要的重建	✓
Q15	综合运用 curl、jq 和 Shell 脚本，实现了从 API 数据到 Markdown 报告的自动化流程	✓
Q16	在限时条件下完成了项目修复、构建和提交的完整流程，提升了综合调试能力	✓

— 报告结束 —

附录 A：Git 提交记录

仓库地址：<https://github.com/ADayxs/SystemDevelopmentTools.git>
本次实验共 4 次提交，均已写入个人 GitHub 仓库（遵守”禁止单次全量提交”要求，按题目分次提交）。提交记录截图如下：



Figure 8: GitHub Commit 记录截图（含 Q13–Q16 各次提交）